

CSR: Infinite-Horizon Real-Time Policies with Massive Cached State Representations

Extended Technical Report for Paper Accepted to IEEE RA-L

Robin Karlsson^{1*,2} and Go Suzui¹

Abstract—Deploying massive large language models (LLMs) as continuous cognitive engines for robotics is bottlenecked by the time-to-first-token (TTFT) latency required to process extensive state histories. Existing solutions like RAG or sliding windows compromise global context or incur prohibitive re-computation costs. We formalize the optimal task structure for minimizing latency and theoretically prove that prefix stability, incremental extensibility, and asynchronous state reconciliation are necessary conditions for real-time performance. Building on these proofs, we introduce the Cached State Representation (CSR) framework as the practical instantiation of these properties, ensuring optimal KV-cache reuse. To sustain these properties over infinite horizons, we further propose an Asynchronous State Reconciliation (ASR) algorithm that offloads state memory eviction to a parallel computational resource to eliminate latency spikes. On a physical robot wirelessly connected to an on-premise GPU server, CSR achieves a 26-fold latency reduction (14.67s to 0.56s) for 120K token contexts with a 235B parameter model compared to a standard baseline. On an embodied AI benchmark, we achieve SOTA recall (0.836 vs. 0.459) while maintaining RAG-level latency. ASR is validated to sustain bounded, spike-free TTFT over 10 eviction cycles in continuous real-world operation. Together, CSR and ASR enable massive LLMs to function as continuously operating, high-frequency (≥ 2 Hz) embodied policies.

I. INTRODUCTION

Large Language Models (LLMs) have emerged as the dominant paradigm for realizing general-purpose robots, providing agents with commonsense reasoning [1]–[3] and open-world generalization capabilities [4]–[6]. To fully leverage these cognitive engines, robots require an extensive state representation that encapsulates static embodiment definitions, continuously updating sensory observations, and accumulated episodic history [7], [8]. However, maintaining such high-fidelity context introduces a fundamental conflict with real-time performance. As the state history grows, the computational cost of processing input sequences grows linearly or quadratically, resulting in prohibitively high time-to-first-token (TTFT) latency [9]. This computational bottleneck typically forces a trade-off between the depth of

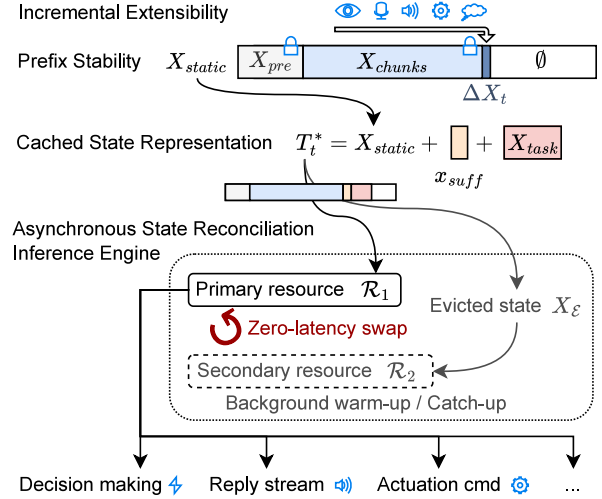


Fig. 1. Overview of the Cached State Representation (CSR) framework for real-time robotic control with massive LLMs. (Top) CSR implements the optimal task structuring strategy S^* by isolating volatile state information into a transient dynamic suffix $x_{suff} + X_{task}$ and appending state information chunks x_t to a static prefix X_{static} . This structure satisfies *Prefix Stability* and *Incremental Extensibility*. (Bottom) The Asynchronous State Reconciliation (ASR) inference engine enables infinite-horizon continuous operation by eliminating eviction latency spikes. A secondary resource $\mathcal{R}_2$ precomputes the KV-cache of the evicted state X_E and synchronizes with the primary resource $\mathcal{R}_1$ via a zero-latency atomic swap.

the agent’s context and its ability to maintain the high-frequency decision-making loops required for responsive interaction [6], [10], [11].

While key-value (KV) caching effectively accelerates text generation in static contexts [12], [13], it is ill-suited for the dynamic state updates inherent to robotics. Standard inference engines operate optimally on append-only sequences. However, robotic high-level policies require continuous state evolution over long time horizons, where new observations are integrated and obsolete data must eventually be evicted due to finite memory. Naively modifying the prompt structure by altering tokens that precede the end of the sequence invalidates the cached attention computations for all subsequent tokens. Consequently, the model is forced to reprocess the entire context history for every inference step, causing latency to scale with context length. Current mitigation strategies, such as Retrieval-Augmented Generation (RAG) [7], [14] or aggressive context pruning [8], [15], circumvent this cost but sacrifice global context awareness, limiting the agent’s ability to execute consistent, long-horizon behavior.

To resolve this conflict, we first formalize the optimal

¹GODOT Inc, Hyogo, Japan

²Graduate School of Informatics, Nagoya University, Aichi, Japan.

*Corresponding author: robin.karlsson@godot.inc

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

task structure S^* for minimizing latency under hardware and information constraints. We theoretically prove two necessary conditions: prefix stability, requiring that historical context remains immutable to preserve cache validity, and incremental extensibility, requiring that new information be strictly appended. Building on these proofs, we introduce the Cached State Representation (CSR) framework as the practical instantiation of S^* , ensuring optimal KV-cache reuse. To sustain these properties over infinite horizons without violating memory limits, we further propose an Asynchronous State Reconciliation (ASR) algorithm that decouples the expensive state eviction process from the primary inference loop. ASR utilizes parallel hardware to precompute the KV-cache for the new state, eliminating latency spikes.

Experiments with a 235B parameter model demonstrate a 26-fold latency reduction (14.67 s to 0.56 s) for 120K token contexts, with ASR validated to sustain bounded, spike-free operation in real-world deployments. Together, CSR and ASR enable massive LLMs to function as continuously operating, high-frequency (≥ 2 Hz) embodied policies without restrictive state memory size. Our specific contributions are:

- A theoretical formalization of the optimal task structure S^* , establishing prefix stability and incremental extensibility as necessary conditions for satisfying real-time deadlines τ_{max} .
- The Cached State Representation (CSR) framework as the practical implementation of S^* , utilizing Asynchronous State Reconciliation (ASR) to enable infinite-horizon real-time operation.
- Empirical validation across varying model sizes (30B–235B) and hardware configurations, demonstrating high-frequency responsiveness (≥ 2 Hz) in continuous real-world deployments and SOTA embodied AI task performance recall (0.836 vs. 0.459) at RAG-level latency, proving both the physical viability and task performance advantage of our framework.

The rest of the paper is organized as follows. Sec. II lists related works. Sec. III presents our formalization and method. Sec. IV and Sec. V explain experiments and results for evaluating performance. Sec. VI positions our findings.

II. RELATED WORKS

LLM applications in robotics: LLMs provides an unprecedented possibility to leverage commonsense knowledge in various robotics tasks. Using LLMs for autoregressive action decision making [1], [16], [17] effectively enhances the versatility and robustness of real-world robots with limited feasibility to learn by trial-and-error. High-level planing [18]–[20] benefits from the remarkable reasoning capability of LLMs. Multimodal reasoning LLMs [4], [21] and Vision-Language-Action (VLA) models [6], [10], [11], [22] enables robots to intelligently assess the robot’s state and predict optimal actions directly from perceptual observations. LLMs can also serve as an internal reward model that allows robots to improve task performance from real-world environment feedback [2], [5], [15], [23]. Coding LLMs can also be used to

iteratively generate highly performant programmatic reward functions [24], [25] and action execution code [3], [26]. Our proposed Cached State Representation (CSR) expands the application domain of massive LLMs in robotics by enabling high frequency action decision making and instantaneous interaction with other agents including humans.

Agentic state representation: A fundamental challenge in LLM-based robotics is how to represent a continuous, 4D spatiotemporal reality into the discrete, sequential latent space required for Transformer architectures. The 3D scene graph approach abstracts geometry into hierarchical semantic nodes [18], [27], [28]. By linearizing these graphs into structured text, LLMs gain a symbolic, queryable summary of the environment. Semantic topological maps represent the environment as searchable, graph-based manifolds where nodes or navigable landmarks consist of localized semantic embeddings [29], [30]. The structured state approach bypass symbolic labeling in favor of directly encode environment information as token sequences with explicit metric and temporal information for fine-grained spatial reasoning. Metric tokenization methods [6], [31], [32] discretizes 3D geometry. Multimodal embedding stream methods [33], [34] creates queryable latent embedding state buffers from encoded high-dimensional sensory percepts. Short-term memory caching approaches attempt to maintain long-horizon agentic consistency within a finite context window. RecurrentGPT [35] implements a recurrent mechanism where the model generates an latent state token embedding. MemGPT [7] utilizes a paging mechanism to swap relevant state information in and out of the token sequence. Closest to our method, StreamingLLM [8] enables infinite-length generation by re-computing the KV cache over a fixed-size sliding window of recent tokens and attention sinks, trading long-horizon memory fidelity for low latency through a short window rather than cache reuse. Long-term episodic memory frameworks utilize external vector databases and retrieval-augmented generation (RAG) to bridge experiences across multiple trials [14], [36], [37]. While effective for lifelong learning, these retrieval-based approaches often introduce significant query latency and context fragmentation, making them ill-suited for real-time behavior generation. Our CSR framework addresses the memory fidelity limitation of StreamingLLM [8] by replacing widow truncation with structured state partitioning, enabling KV cache reuse while retaining the full state history up to $\mathcal{O}(10^5)$ tokens, with latency scaling linearly in sequence length.

Real-Time and efficient LLM inference: Continuously operating robots performing tasks relying on real-time interaction with the environment requires low latency and high throughput LLM inference engines. Model compression techniques, such as weight quantization [38], [39] and low-precision architectural paradigms [40], reduce the memory and computational overhead of LLMs with limited performance degradation. Speculative decoding and parallel sampling strategies [41], [42] speeds up generation by using small draft models to predict multiple tokens each forward pass. Innovations in efficient attention computa-

tion mitigates the quadratic memory complexity limiting very long states by hardware-optimized kernel [43] and distributed computation strategies [44]. Recent LLM serving frameworks implement KV-cache optimizations including dynamic memory orchestration via paged allocation [12] and continuous batching [45] to eliminate VRAM fragmentation and maximize inference throughput. Semantic context reuse strategies like RadixAttention [46] and prefix caching [13] allow for the instantaneous reuse of pre-computed token sequences. KV-Cache compression frameworks [9], [47] minimize the memory footprint and transmission latency of state information shared over a network. Our proposed CSR framework extends these serving primitives by providing a theoretical framework for optimal state structuring for KV-cache optimization frameworks like vLLM [12] to ensure minimal latency, and an ASR algorithm that together with CSR ensures prefix stability and zero-latency state eviction for continuous, high-frequency robotic control loops. While CSR does not reduce continuous token output generation speed, CSR is compatible with and complements existing methods for this purpose [41].

III. CACHED STATE REPRESENTATION

In this section we present how intermediate computational states are cached and reused to speed up single Transformer-based LLM inference tasks. Thereafter we show how to extend cache reuse to sequential requests via the Cached State Representation (CSR) framework, and how the Asynchronous State Reconciliation (ASR) algorithm eliminates eviction-induced latency spikes to sustain this property over infinite horizons, achieving minimal *time-to-first-token* (TTFT) for high-frequency robot AI inference.

A. Background: Transformer Architecture and KV Caching

The Transformers architecture is a general-purpose model of computation that excels in learning the probability distribution of sequences of discrete elements

$$X = (x_1, \dots, x_N) \in \mathcal{X}^N \quad (1)$$

by autoregressive decomposition of the joint probability of a sequence:

$$P(X) = P(x_1, \dots, x_N) \quad (2)$$

$$= P(x_n | x_1, \dots, x_{n-1}) \dots P(x_2 | x_1) P(x_1). \quad (3)$$

Current LLMs are variations of the decoder-only Transformer architecture and model token sequence probabilities. A token represent a substring x from the finite set $\mathcal{X}$ containing K substrings from which all possible strings in a language can be created. An LLM predicts a discrete probability distribution $y \in \mathbb{R}^K$ for the next token x_{n+1} by first mapping the discrete token sequence X into a latent token embedding matrix $H^{(0)} = [h_1^{(0)}, \dots, h_n^{(0)}]$ with embedding vectors $h \in \mathbb{R}^{d_h}$. Next, L sequential decoder blocks recurrently updates $H^{(\ell-1)}$ to $H^{(\ell)}$ by a self-attention

module and feed forward networks (FFN):

$$k_n^{(\ell)} = W_K^{(\ell)} h_n^{(\ell-1)} \in \mathbb{R}^{1 \times d_k} \quad (4)$$

$$v_n^{(\ell)} = W_V^{(\ell)} h_n^{(\ell-1)} \in \mathbb{R}^{1 \times d_v} \quad (5)$$

$$q_n^{(\ell)} = W_Q^{(\ell)} h_n^{(\ell-1)} \in \mathbb{R}^{1 \times d_k} \quad (6)$$

$$a_n^{(\ell)} = \text{softmax} \left(\left[\frac{q_n^{(\ell)} \cdot k_{\tau}^{(\ell)}}{\sqrt{d_k}} \right]_{\tau=1}^n \right) \quad (7)$$

$$= \text{softmax} \left(\frac{q_n^{(\ell)} [\mathcal{K}_{1:n}^{(\ell)}]^T}{\sqrt{d_k}} \right) \in \mathbb{R}^n \quad (8)$$

$$o_n^{(\ell)} = \sum_{\tau=1}^n a_{n,\tau}^{(\ell)} v_{n,\tau}^{(\ell)} = a_n^{(\ell)} [\mathcal{V}_{1:n}^{(\ell)}] \in \mathbb{R}^{d_v} \quad (9)$$

$$z_n^{(\ell)} = \text{LN}(h_n^{(\ell-1)} + W_O^{(\ell)} o_n^{(\ell)}) \quad (10)$$

$$h_n^{(\ell)} = \text{LN} \left(z_n^{(\ell)} + \text{FFN}^{(\ell)} \left(z_n^{(\ell)} \right) \right) \quad (11)$$

$$y = \text{softmax}(W_Y H^{(L)}) \quad (12)$$

$$x_{n+1} = f(y) \quad (13)$$

where

$$[\mathcal{K}_{1:n}^{(\ell)}] = [k_1^{(\ell)}, \dots, k_n^{(\ell)}] \quad (14)$$

$$[\mathcal{V}_{1:n}^{(\ell)}] = [v_1^{(\ell)}, \dots, v_n^{(\ell)}] \quad (15)$$

denotes cached concatenations of keys and value vectors for tokens 1 to n . Caching key and value vectors as in (14)-(15) effectively amortizes or reuses the computation of intermediate computational states for tokens 1 to n , as those states are not altered by subsequent tokens due to the causal attention mechanism.

Utilizing KV caching is a standard technique for reducing latency when processing an independent growing token sequence X representing a single inference task T . In this paper, we consider robot AI inference tasks such as decision making, reply, and actuation command generation. However, high-frequency embodied robot AI tasks are not independent, but sequential and conditional on a subset of all available information in the robot system.

In the next section, we explain why naively structured sequential inference tasks generally breaks the cache, resulting in high TTFT latency for robot AI applications.

B. KV Cache Reuse Across Sequential Tasks

We formalize a robot AI inference task T as

$$T = (X_{in}, X_{out}) = [x_1^{in}, \dots, x_N^{in}, x_{N+1}^{out}, \dots, x_M^{out}], \quad (16)$$

where X_{in} and X_{out} are the input and output token sequences. X_{in} contains the task specification and information required to complete the task. X_{out} is initially an empty set $\emptyset$ and is appended with new tokens generated through (13).

When starting to process a new task

$$T' = (X'_{in}, \emptyset) = [\hat{x}_1^{in}, \dots, \hat{x}_S^{in}], \quad (17)$$

the cached KV cache vectors (14)-(15) for all tokens in (16) becomes invalid at the first token in T' differing with the corresponding token in T :

$$i^* = \min\{i \in \{1, \dots, S\} : T_i \neq T'_i\}. \quad (18)$$

In particular, if the first token $i = 1$ in (18) for T' is different from T , the entire KV cache for X'_{in} needs to be recomputed from scratch. Note that this behavior is expected and unavoidable for general independent tasks T and T' .

In the next section we formalize the sequential robot AI inference task objective, and derive required properties for an optimal task representation.

C. Optimal Task Structure

To derive an optimal solution to the problem explained in Sec. III-B, we first formalize the optimization objective for sequential robot AI inference tasks as

$$\min_S \max_t \text{TTFT}(T_t|S). \quad (19)$$

The objective (19) is subject to the following constraints:

$$\text{TTFT}(T_t|S) \leq \tau_{max} \quad \forall t \quad (20)$$

$$\text{Info}(T_t|S) \supseteq \mathcal{I}_{req}^{(t)} \quad \forall t \quad (21)$$

$$\text{Memory}(T_t|S) \leq M_{max} \quad \forall t \quad (22)$$

where S is a task structuring strategy from the set of all possible strategies $\mathcal{S}$. τ_{max} is maximum allowed deadline for real-time inference. $\text{Info}(T_t)$ is the information contained in the task representation T_t and must contain all information required to complete the task $\mathcal{I}_{req}^{(t)}$. M_{max} represents the finite memory in which T_t can be stored.

Next we derive TTFT as a function of cache reuse. From equations (4)-(13), the computational cost for processing task T_t is

$$\text{TTFT}(T_t) = \sum_{i=i^*}^{|T_t|} \sum_{\ell=1}^L C \cdot f_{ops}(i, \ell), \quad (23)$$

where C is the computational cost per operation and $f_{ops}(i, \ell)$ is the number of operations for token i at layer ℓ . For standard Transformers utilizing KV caching, the number of operations is a linear function of latent embedding dimensionality d_h and the number of preceding tokens due to the causal attention mechanism (7)-(9):

$$f_{ops}(i, \ell) = \mathcal{O}(i \cdot d_h). \quad (24)$$

Therefore (23) can be simplified as a linear function of i :

$$\text{TTFT}(T_t) \propto L d_h \sum_{i=i^*}^{|T_t|} i = L d_h \frac{(|T_t| - i^* + 1)(|T_t| + i^*)}{2}. \quad (25)$$

It follows that minimizing TTFT requires maximizing $i^*(T_{t-1}, T_t)$ for sequential tasks T_t and T_{t+1} according to (18).

Theorem 1: [Prefix Stability] To satisfy the real-time constraint (20) for sequential tasks with bounded computation, sufficient prefix stability is necessary. The task structure S must ensure that the first differing token index i^* is greater than or equal to a minimum stable prefix length i_{min}^* :

$$i^*(T_t, T_{t+1}) \geq i_{min}^* \quad \forall t, \quad (26)$$

where i_{min}^* is defined as the minimum prefix match i^* such that the computational cost N_{FLOPs} for the new task T_{t+1} satisfies the real-time budget B_{max} :

$$N_{FLOPs}(T_{t+1}|i^* = i_{min}^*) \leq B_{max}. \quad (27)$$

An optimal strategy S^* minimizes (19) by structuring sequential tasks T_t to achieve optimal prefix stability:

$$S^* \implies i^*(T_t, T_{t+1}) = |T_t| + 1 \quad \forall t. \quad (28)$$

Proof: See Appendix A. ■

The proof is based on estimating the required FLOPs without prefix stability to be $\mathcal{O}(100)$ higher than with prefix stability.

Theorem 2 (Incremental Extensibility): For a robot AI system where the total information context is non-decreasing ($\text{Info}(T_{t+1}|S) \supseteq \text{Info}(T_t|S)$), any task structuring strategy S must be incrementally extensible by a state increment ΔX_t

$$T_{t+1} = T_t \oplus \Delta X_t \quad (29)$$

in order to satisfy the real-time constraint (20) and information sufficiency constraint (21) for all tasks in a sequence (T_t) .

Proof: See Appendix B. ■

The proof shows by contradiction that any S not following Theorem 2 violates Theorem 1.

Theorem 3: [Eviction] Given the incremental extensibility from Theorem 2, any strategy S for infinite-horizon operation ($t \rightarrow \infty$) must include an eviction operation $\mathcal{E}$ to ensure the memory constraint (22) is satisfied:

$$T_{t+1} = \mathcal{E}(T_t \oplus \Delta X_t) \quad (30)$$

Proof: See Appendix C. ■

The proof by contradiction demonstrates that the unbounded growth of T_t as $t \rightarrow \infty$ according to Theorem 2 and inevitably violates (22).

Theorems 1 and 3 creates a fundamental tension, as applying the eviction operation (30) violates token sequence order required for prefix stability (26). Theorem 4 proposes a solution to satisfy Theorems 1-3 simultaneously by performing asynchronous state reconciliation in parallel on a secondary computation hardware.

Theorem 4: [Asynchronous State Reconciliation] To satisfy the real-time constraint (20) (Theorem 1) during an eviction operation (30) (Theorem 3), the KV cache computation for the new evicted state $T_t^\mathcal{E}$ must be offloaded to a parallel computational resource.

Continuous real-time operation is achieved by:

- 1) Warming up $T_t^\mathcal{E}$ on the secondary resource.
- 2) Reconciling $T_t^\mathcal{E}$ with the new token sequence ΔX_t processed by the primary resource during warm-up.
- 3) Swapping the primary state T_t with the reconciled state $T_t^\mathcal{E}$ before T_t violates the memory constraint (22).

Proof: See Appendix D. ■

The proof is based on showing the eviction process must be offloaded to satisfy Theorem 1, and construct a method and derive an inequality constraining viable free variables.

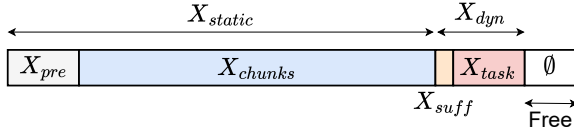


Fig. 2. Structure of the Cached State Representation (CSR). The context is partitioned into a reusable static prefix X_{static} and a transient dynamic suffix X_{dyn} .

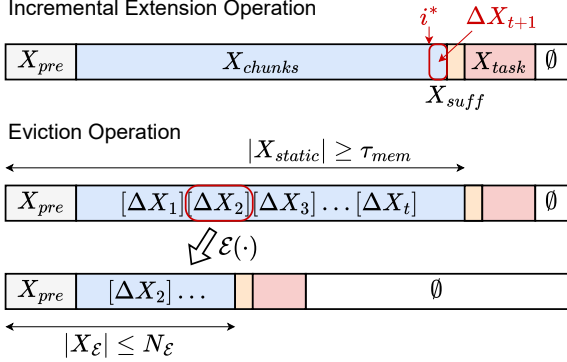


Fig. 3. CSR operations. (Top) Incremental Extension appends x_{t+1} while maintaining cache. (Bottom) Eviction removes older chunks to satisfy memory constraints.

Corollary 1: [Optimal Strategy] A task structure strategy S that satisfies Theorem 1-4 is the optimal strategy S^* for optimizing (19) while satisfying the constraints (20)-(22).

Proof: See Appendix E. ■

The proof is based on mapping the optimization of (19) and constraints satisfaction (20)-(22) to each theorem.

In the next section we present the Cached State Representation (CSR) framework as the practical task representation structure strategy that is closest to S^* .

D. Cached State Representation (CSR) Framework

In this section we introduce our Cached State Representation (CSR) framework as the concrete task representation implementing the optimal strategy S^* derived in Sec. III-C. We derive CSR as the direct embodiment of the Prefix Stability (Theorem 1) and Incremental Extensibility (Theorem 2) properties of S^* :

First, a task structure T_t that satisfies Theorem 1 must guarantee a minimal stable prefix i_{min}^* in (26). We achieve this by partitioning the robot AI's state into a static state prefix X_{static} containing immutable information. Static information include robot profile, agency and behavior information, in-context learning information, action descriptions, etc. We call this set of information the state prefix X_{pre} . Additionally, all information accumulated during robot AI operation including observational experience, internal thoughts, and action execution output, etc., are stored as state chunks x_t in an ordered sequence $X_{chunks} = (x_0, \dots, x_t)$. The KV cache for

$$X_{static} = (X_{pre}, X_{chunks}) \quad (31)$$

is computed once and guarantees $i^* \geq |X_{static}|$.

Secondly, to satisfy Theorem 2, T_t must be incrementally extensible (29). We implement this by appending new state

information x_t as a sequence of state chunks which accumulates all information the robot AI obtains during operation such as observational experience, internal thoughts, and action execution output, etc. Continuously appending x_t while performing high-frequency robot AI inference tasks allows the KV cache computations (4)-(5) to be amortized across many small state chunk subsets $\Delta X_{chunks} = (x_{t+1}, \dots, x_T)$ so that i^* in (18) is close to the optimal prefix stability (28).

Practical general-purpose robot AI systems also need to consider dynamic information X_{dyn} representing non-sequential information: First, an information component where to write instantaneous information lacking historical value, such as current time, to avoid cluttering X_{chunks} . We call this component the state suffix X_{suffix} . Secondly, a component X_{task} where to specify the current task instruction, such as decision making or reply generation. Due to its dynamic nature, the KV cache for

$$X_{dyn} = (X_{suffix}, X_{task}) \quad (32)$$

is generally recomputed for every task and should be as small as possible.

The resulting CSR structure T_t^* is illustrated in Fig. 2 and formalized as the concatenation of all aforementioned information components:

$$T_t^* = (X_{static}, X_{dyn}) \quad (33)$$

$$|X_{static}| \gg |X_{dyn}| \quad (34)$$

$$i^* = |X_{static}| + 1. \quad (35)$$

Figure 3 illustrate the incremental extension and eviction operations on T_t^* . Note that T_t^* approaches S^* as $|X_{dyn}| \rightarrow 0$. In the next section we present a practical implementation of the reconciliation method based on Theorem 4.

E. Asynchronous State Reconciliation Algorithm

Theorem 4 necessitates a state reconciliation method to overcome the fundamental tension between Theorems 1 (stable prefix) and 3 (eviction). We present the asynchronous state reconciliation method in Algorithm 1, with the process flow illustrated in Fig. 4. This implementation realizes Theorem 4 and follows the constructive proof provided in Appendix D.

The method bifurcates the inference process into two concurrent execution tracks: an active primary resource $\mathcal{R}_1$ responsible for serving robot AI inference tasks, and a secondary resource $\mathcal{R}_2$ used for asynchronously reconciling the evicted state X_E in the background. When $\mathcal{R}_1$ reaches a memory threshold τ_{mem} , it triggers the eviction operation (30) and begins recomputing the KV cache (14)-(15) (i.e., warmup) on $\mathcal{R}_2$ (Lines 10-14). During this phase, $\mathcal{R}_1$ continues to serve inference tasks in parallel, while new incoming state chunks ΔX_t are simultaneously buffered in X_{buffer} (Lines 6-8). Once the initial KV cache is recomputed, $\mathcal{R}_2$ iteratively updates the cache with the buffered chunks (i.e., catch-up) until the buffer is drained (Lines 16-20). Upon convergence, an atomic swap promotes $\mathcal{R}_2$ to the primary

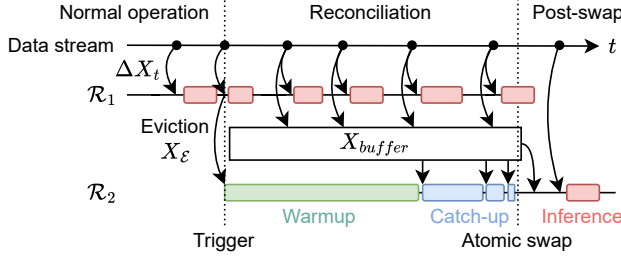


Fig. 4. Asynchronous reconciliation process timeline. $\mathcal{R}_1$ serves real-time inference queries in parallel while $\mathcal{R}_2$ warms up on the evicted state X_ε and integrates buffered chunks X_{buffer} during catch-up. Upon synchronization, an atomic swap promotes $\mathcal{R}_2$ to primary, maintaining low-latency operation.

resource (Lines 22-23). When $\mathcal{R}_2$ eventually fills, the reconciliation process is repeated on $\mathcal{R}_1$. The method ensures continuous real-time operation without latency spikes.

IV. EXPERIMENTS

We evaluate the proposed CSR framework on four distinct objectives: (1) quantifying the Time-to-First-Token (TTFT) reduction, (2) assessing the hardware generalization, (3) validating the stability of the ASR algorithm, and (4) benchmark embodied AI task performance.

A. Experimental Setup

Model and hardware: Unless otherwise noted, experiments utilize the *Qwen 2.5-235B-Instruct-FP8* model served via a vLLM inference server with all caching features enabled on 4×H100 GPUs. We utilize *Qwen 2.5-32B-Instruct* for single GPU ablation studies. To assess real-world performance, we deploy the CSR framework on a physical robot. The system is built in ROS 2 Jazzy and runs on a Raspberry Pi 5. The robot is connected to an on-premise GPU server by a wireless network, reflecting the emerging mobile edge computing paradigm enabled by post-5G infrastructure [48].

Prompt construction: To emulate the CSR structure defined in Sec. III-D, we construct input prompts T as a concatenation of a static prefix X_{static} and a dynamic suffix X_{dyn} :

$$T = X_{static} \oplus X_{dyn}, \quad |X_{static}| = N, \quad |X_{dyn}| = M. \quad (36)$$

X_{static} is sampled from random text with a fixed seed to ensure cache hits, while X_{dyn} uses a random seed to enforce recomputation of the corresponding KV cache.

Metric: We report TTFT, measuring the duration from request receipt to the generation of the first token. Each data point represents the mean and standard deviation over 5 independent trials.

B. Evaluation Scenarios

Latency analysis: We sweep the static context length $N \in [1, 120K]$ and dynamic suffix length $M \in \{1, \dots, 8192\}$. We compare CSR against a naive unordered baseline where the entire prompt T is treated as dynamic (unordered), forcing a full cache re-computation at every step. This quantifies the overhead of X_{task} size on real-time performance.

GPU comparison: To assess deployment feasibility, we evaluate CSR with $N \in \{30K, \dots, 120K\}$ across varying hardware configurations.

Algorithm 1 Asynchronous State Reconciliation

Require: Stream of state chunks $\{\Delta X_t\}$

Require: Parameters: τ_{mem} , $N_{catchup}$

State Variables:

$X_{static} \leftarrow X_{pre}$ // Current static state prefix
 $\Delta X_{buffer} \leftarrow \emptyset$ // State chunk buffer
 $X_\varepsilon \leftarrow \emptyset$ // Evicted/compressed state
 $r_{primary} \leftarrow \mathcal{R}_1$ // Currently active resource
 $r_{secondary} \leftarrow \mathcal{R}_2$ // Resource being warmed up
 $is_reconciling \leftarrow false$ // Status flag

1: **Procedure** INCREMENT(ΔX_t):

2: // Always update the active primary state

3: $X_{static} \leftarrow X_{static} \oplus \Delta X_t$

4:

5: // Fork data if a background reconciliation is active

6: **if** $is_reconciling$ **then**

7: $\Delta X_{buffer} \leftarrow \Delta X_{buffer} \oplus \Delta X_t$

8: **end if**

9:

10: **if** $|X_{static}| \geq \tau_{mem}$ **and not** $is_reconciling$ **then**

11: $X_\varepsilon \leftarrow \text{Evict}(X_{static})$

12: Launch RECONCILE() in parallel // Asynchronous

13: $is_reconciling \leftarrow true$

14: **end if**

15:

16: **Procedure** RECONCILE():

17: // Warm up secondary resource with evicted state

18: LLM($X_\varepsilon, r_{secondary}$)

19:

20: // Reconcile buffered chunks until sufficiently caught up

21: // NOTE: Reminder is computed at next task inference

22: **while** $|\Delta X_{buffer}| > N_{catchup}$ **do**

23: $X_\varepsilon \leftarrow X_\varepsilon \oplus \Delta X_{buffer}$

24: $\Delta X_{buffer} \leftarrow \emptyset$

25: LLM($X_\varepsilon, r_{secondary}$)

26: **end while**

27:

28: // Swap primary and secondary resource pointers

29: $r_{primary}, r_{secondary} \leftarrow r_{secondary}, r_{primary}$

30: $is_reconciling \leftarrow false$

Infinite-horizon low-latency operation: We validate Algorithm 1 by running a ROS 2 implementation on a physical robot system. The robot receives a continuous stream of state updates ΔX_t each containing 100 tokens. The robot runs on two *Qwen 2.5-32B-Instruct* models; one model is used for the robot policy, and the other model is used for state reconciliation onset by a state eviction process. Eviction is triggered at a memory threshold $\tau_{mem} = 110K$ tokens, and discards the oldest half of the elements in X_{chunks} . We measure full end-to-end LLM response cycle time and detect latency spikes by TTFT following how our robot policy predicts action decisions as single tokens. As our CSR framework reduce latency, not time for sequential output token generation, we refer to dedicated work on output

generation speed for LLM-based robot policies for further application analysis. See Appendix F for the pseudo-code describing the ASR implementation supporting concurrent requests and used on the physical robot system.

Embodied AI task performance: We evaluate CSR embodied AI task performance on AMA-bench [37]. This benchmark measures long-horizon memory fidelity across continuous agent-environment interactions. We compare performance against three competitive baselines representing the existing paradigms. First, StreamingLLM [8] utilizing sliding attention windows representing kernel-level compute amortization. Secondly, BM25 representing a standard RAG implementation. Finally, AMA-Agent [37], a SOTA memory system leveraging causality graphs and tool-augmented retrieval. We measure latency and jitter to characterize real-time inference capability of each method. Method performance is primarily estimated with the *Qwen3.5-122B-A10B-FP8* models as the inference backbone. We also evaluate on the *Qwen3-30B-A3B* model that is supported in TensorRT-LLM to serve StreamingLLM.

V. RESULTS

Latency analysis: The primary bottleneck for real-time robotics is TTFT. As shown in Fig. 5 and Table II, CSR achieves a radical performance improvement over the unordered baseline. For a 120K token context, CSR reduces latency from 14.67s to 0.56s (26-fold reduction). While latency scales linearly with dynamic suffix length M , CSR maintains sub-second response times < 0.6 s even with significant dynamic instructions $M = 1024$. This indicates that the framework can accommodate substantial task-specific prompt engineering and large immediate observations without breaking the real-time loop. These results provide direct empirical evidence that the optimal task structure formalized by CSR is strictly necessary to achieve manageable latency and avoid latency spikes. CSR thereby makes high-frequency control possible with massive LLMs on current general-purpose hardware.

GPU comparison: Table I illustrate CSR’s performance across different hardware. Notably, the workstation-class RTX PRO 6000 achieves the lowest latency for moderate dynamic loads ($M \approx 1024$), outperforming data-center GPUs. This suggests that on-premise workstations are a viable and cost-effective platform for deploying massive embodied agents. However, for larger dynamic updates ($M \geq 4096$), the PCIe communication bandwidth likely becomes a bottleneck.

Infinite-horizon low-latency operation: Fig. 6 shows that CSR combined with the ASR algorithm (bottom) successfully eliminates the latency spike associated with an eviction operation in continuous operation over 30 minutes and 10 reconciliation cycles without introducing overhead. Figure 7 visualizes the latency distribution and jitter to characterize real-time predictability and validate our high-frequency control claims. The density distribution (left) shows that TTFT remains bounded with a mean latency of 0.347 s and 99th percentile (p99) capped at 0.520 s. The box plot (right)

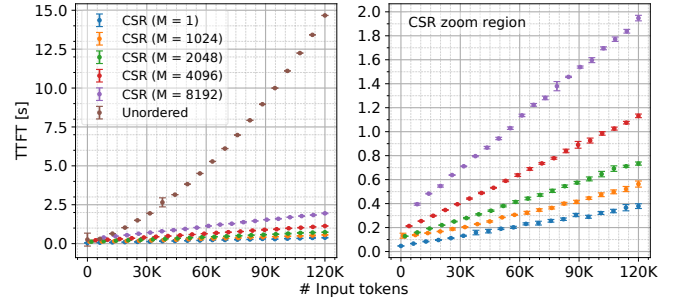


Fig. 5. Latency experiment results. The left figure shows how prompts structured according to CSR radically reduces latency or time-to-first-token (TTFT) for long states and massive 235B model compared with generic unordered state prompts. The right figure display how larger dynamic state suffixes x_{suff} and x_{task} of token length M penalize latency.

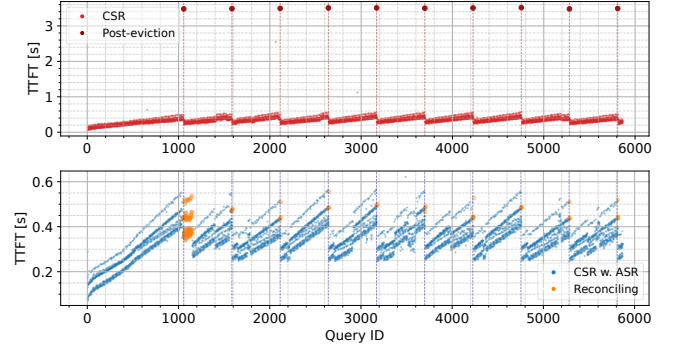


Fig. 6. CSR with ASR (bottom) avoids latency spikes after a state eviction operation and infinite horizon low-latency operation.

demonstrates the long-horizon stability of the ASR process. The TTFT variance and median response lacks measurable drift across 10 successive state eviction cycles, backing our theoretical claims of perpetually stability and high-frequency operation.

Embodied AI task performance: Table III shows the CSR framework successfully supports high-frequency operation without sacrificing extensive context required for complex embodied tasks. The 30B model experiments demonstrate how StreamingLLM with a vanilla attention window size of 4096 tokens cannot represent long-horizon state memory. The maximum supported length of 32,768 tokens captures sufficient context but is slow due to having to recompute the KV cache for the large window at every state update. For the more capable 122B model, CSR providing the complete state information achieves a SOTA recall of 0.836, substantially outperforming both the AMA agent with a graph-based memory system (0.459) and the BM25 RAG baseline (0.295) while maintaining a highly predictable, low-latency profile with minimal jitter (TTFT mean 0.220s, std 0.052s) similarly to the RAG method.

VI. CONCLUSIONS

We presented a theoretical and empirical investigation of state structure and the latency bottleneck in LLM-based embodied intelligence. By formalizing the sequential robot AI inference problem, we proved that Prefix Stability, Incremental Extensibility, and Asynchronous State Reconciliation are necessary conditions for continuous real-time operation even with massive models. The CSR framework instantiates

TABLE I
GPU PERFORMANCE COMPARISON (TTFT IN SECONDS, MEAN)

Input Length M Value	30K			60K			90K			120K		
	M=1	M=1024	M=4096	M=1	M=1024	M=4096	M=1	M=1024	M=4096	M=1	M=1024	M=4096
4x H100 SXM5*	0.13	0.20	0.41	0.22	0.31	0.65	0.30	0.42	0.89	0.38	0.56	1.13
H100 SXM5	0.13	0.18	0.33	0.22	0.30	0.58	0.30	0.41	0.80	0.42	0.57	1.06
GH200	0.11	0.16	0.31	0.19	0.27	0.54	0.27	0.37	0.76	0.36	0.49	0.99
H100 PCIe	0.29	0.32	0.61	0.45	0.51	1.02	0.68	0.71	1.45	0.78	0.88	1.87
RTX PRO 6000	0.15	0.15	0.51	0.22	0.24	0.90	0.29	0.32	1.30	0.39	0.43	1.73

TABLE II
LATENCY RESULTS (MEAN $\pm$ STD IN SECONDS)

	30K	60K	90K	120K
Unord.	1.83 $\pm$ 0.00	4.90 $\pm$ 0.01	9.22 $\pm$ 0.01	14.67 $\pm$ 0.01
M=1	0.12 $\pm$ 0.00	0.21 $\pm$ 0.02	0.27 $\pm$ 0.01	0.34 $\pm$ 0.01
M=1024	0.20 $\pm$ 0.01	0.31 $\pm$ 0.01	0.42 $\pm$ 0.01	0.56 $\pm$ 0.03
M=2048	0.26 $\pm$ 0.00	0.42 $\pm$ 0.01	0.58 $\pm$ 0.01	0.73 $\pm$ 0.01
M=4096	0.41 $\pm$ 0.00	0.65 $\pm$ 0.01	0.89 $\pm$ 0.03	1.13 $\pm$ 0.02
M=8192	0.69 $\pm$ 0.00	1.11 $\pm$ 0.01	1.53 $\pm$ 0.01	1.95 $\pm$ 0.02

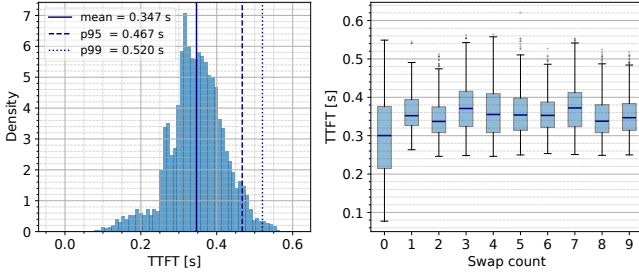


Fig. 7. CSR with ASR maintains a bounded (left) and consistent (right) TTFT distribution over 10 eviction cycles.

these conditions as a practical task structuring strategy, enabling optimal KV-cache reuse across sequential inference tasks without modifying the underlying model or inference engine. To sustain these properties over infinite horizons, the Asynchronous State Reconciliation (ASR) algorithm offloads state eviction to a secondary compute resource, eliminating the latency spikes that would otherwise periodically violate real-time deadlines. Empirical validation on a physical robot with a 235B parameter model confirms a 26-fold TTFT reduction at 120K token contexts, spike-free operation across 10 eviction cycles, and SOTA embodied task recall of 0.836, nearly double the next best system by retaining the complete state context while maintaining RAG-level latency. Together, these results demonstrate that wirelessly connected on-premise servers running massive cognitive engines are a viable architecture for high-frequency embodied AI, and that the path to capable real-time agents runs through principled state representation rather than model compression or context truncation.

APPENDIX

A. Proof for Theorem 1

Proof: The total number of floating-point operations N_{FLOPs} to complete a task T_t with the real-time constraint (20) requires

$$N_{FLOPs}(T_t) \leq \tau_{max} F_{hw}, \quad (37)$$

TABLE III
AMA-BENCH: EMBODIED AI DOMAIN

Model	Method	Embodied AI	
		TTFT mean (std) [s]	Recall
30B	StreamingLLM (4096*) [8]	0.228 (0.0302)	0.000
	StreamingLLM (32,768*) [8]	2.674 (1.084)	0.180
	BM25	0.137 (0.0402)	0.066
	AMA-Agent [37]	2.255 (1.960)	0.066
	CSR (ours)	0.179 (0.0460)	0.115
122B	BM25	0.2059 (0.1027)	0.295
	AMA-Agent [37]	0.9262 (0.287)	0.459
	CSR (ours)	0.220 (0.052)	0.836

*: Sliding attention window size [tokens]

where F_{hw} is the hardware's FLOPs. We define

$$B_{max} = \tau_{max} F_{hw} \quad (38)$$

as the maximum computational budget corresponding to (37). We construct a proof by contradiction by assuming prefix stability is not necessary to satisfy (37) and (38) with $i^* = 1$ in (18):

$$N_{FLOPs}(T_t^{i^*=1}) \leq B_{max} \quad (39)$$

The number of FLOPs is proportional to the number of tokens estimated in (25):

$$N_{FLOPs}(T_t^{i^*=1}) \propto \frac{1}{2} L d_h |T_t|^2. \quad (40)$$

Estimating the number of FLOPs for sequential tasks with prefix stability with a small token addition $\Delta \ll |T_t|$ gives

$$N_{FLOPs}(T_t^{i^*=|T_t|-\Delta}) \propto \frac{1}{2} L d_h \Delta |T_t| \quad (41)$$

The ratio of (40) and (41) is

$$\frac{N_{FLOPs}(T_t^{i^*=1})}{N_{FLOPs}(T_t^{i^*=|T_t|-\Delta})} \approx \frac{|T_t|}{\Delta}. \quad (42)$$

In this work, we consider $|T_t| \sim \mathcal{O}(10^5)$ tokens per task and small token increments $\Delta \sim \mathcal{O}(10^3)$. Accordingly, (42) estimates

$$\frac{N_{FLOPs}(T_t^{i^*=1})}{N_{FLOPs}(T_t^{i^*=|T_t|-\Delta})} \approx \frac{10^5}{10^3} = 10^2. \quad (43)$$

Latency for LLMs with very large input token sequences is a recognized problem and experimentally verified in Fig. 5, and lower latency is advantageous for a real-time robot AI system. Suppose it is possible to achieve satisfactory high-frequency operation by utilizing prefix stability such that

$$N_{FLOPs}(T_t^{i^*=|T_t|-\Delta}) \leq B_{max}. \quad (44)$$

However, if latency without stable prefix $N_{FLOPs}(T_t^{i^*=1})$ is $\mathcal{O}(100)$ times higher than when using stable prefix, it is a computational certainty that

$$N_{FLOPs}(T_t^{i^*=1}) \gg B_{max}. \quad (45)$$

Therefore, the assumption (39) is false, meaning prefix stability is necessary to satisfy the real-time constraint (44) for large input token sequences. ■

B. Proof for Theorem 2

Proof: Let S be a task structuring strategy that satisfies constraints (20) and (21). Theorem 1 determines that S must ensure prefix stability to satisfy (20). S implementing the concatenation operation $\oplus$ in (29) ensures optimal prefix stability by achieving $i^* = |T_t| + 1$, and is the only optimal structure.

S satisfies the information sufficiency constraint (21) based on the premise that the total information context is non-decreasing

$$\text{Info}(T_{t+1}|S) \geq \text{Info}(T_t|S) + \text{Info}(\Delta X_t). \quad (46)$$

This ensures that any arbitrary task requirement can be met:

$$\mathcal{I}_{req}^{(t+1)} \subseteq \text{Info}(T_{t+1}|S). \quad (47)$$

Next, we prove that the necessity of Theorem 2 by contradiction. Assume that S is not incrementally extensible. This means S does not adhere to the structure in (29). Thus S must modify T_t in a way other than appending (e.g., insertion, deletion, or replacement) at an index $i \leq |T_t|$ to encode the required information $\text{Info}(T_{t+1}|S)$. By the definition of i^* in (18), any such modification forces $i^*(T_t, T_{t+1}) \leq |T_t|$. Smaller i^* values increasingly violates the condition for sufficient prefix stability specified in Theorem 1.

This result contradicts the assumption that S need not be incrementally extensible as defined in (29). ■

C. Proof for Theorem 3

Proof: We prove the necessity of eviction by contradiction. Assume a strategy S exists for infinite-horizon operation that satisfies the constraints (20)-(22) without an eviction operation $\mathcal{E}$. Satisfying (20) and (21) means S must be incrementally extensible according to Theorem 2. The length of task sequence $|T_t|$ is therefore monotonically non-decreasing:

$$|T_{t+1}| = |T_t| + \Delta X_t \geq |T_t|. \quad (48)$$

For any non-trivial robot AI system, new information is continuously processed:

$$\mathbb{E}[|\Delta X_t|] > 0. \quad (49)$$

The total sequence length thus grows without bound:

$$|T_t| = |T_0| + \sum_{\tau=0}^{t-1} |\Delta X_\tau| \implies \lim_{t \rightarrow \infty} |T_t| = \infty. \quad (50)$$

Assuming memory cost is proportional to sequence length: $|T_t| \propto \text{Memory}(T_t|S)$, there will exist a time t_{OOM} such

that for all $t \geq t_{OOM}$ the finite memory constraint (22) is violated.

Therefore, the initial assumption is false. Any viable S for infinite-horizon operation must include an operation $\mathcal{E}$ as in (30) to prevent unbounded growth by removing information. ■

D. Proof for Theorem 4

Proof: We prove that the eviction processes must be offloaded by contradiction. Suppose a task representation T_t satisfies Theorem 1 with $i^* \approx |T_t|$. Next an eviction operation (30) is performed synchronously on T_t . Any non-trivial eviction operation necessarily modifies the prefix structure so that $i^* \ll |T_t|$, and thus violate (26) in Theorem 1, meaning real-time operation is not possible. Therefore, synchronous in-place eviction is not feasible and the operation must be offloaded.

Next, we create a proof by construction for the asynchronous state reconciliation processes:

- 1) Duplicate $T_t \rightarrow \hat{T}_t$ when $|T_t|$ reaches a memory threshold $\tau_{mem} < N_{max}$ at time t_{evict} .
- 2) Apply the eviction operation on the copy:

$$T_t^\mathcal{E} = \mathcal{E}(\hat{T}_t). \quad (51)$$

- 3) Warm up $T_t^\mathcal{E}$ by recomputing the KV cache on the secondary hardware resource $\mathcal{R}_2$.
- 4) The primary hardware resource $\mathcal{R}_1$ continue to process tasks and append new tokens $\Delta X_{t_{evict}:T}$ accumulated during the warm-up.
- 5) Once $T_t^\mathcal{E}$ has completed warm-up and is synced or reconciled:

$$T_T^\mathcal{E} \equiv \mathcal{E}(T_t) \oplus \Delta X_{t_{evict}:T}, \quad (52)$$

the pointer for T_T in $\mathcal{R}_1$ is swapped to $T_T^\mathcal{E}$ in $\mathcal{R}_2$:

$$T_T := T_T^\mathcal{E}. \quad (53)$$

The new sequential task T_{T+1} is now an incremental extension of the evicted task $\mathcal{E}(T_t)$ loaded in $\mathcal{R}_2$.

The processes is viable if and only if the total time for the parallel work (e.g., warm-up and reconciliation) is less than the time to memory overflow failure on $\mathcal{R}_1$:

$$\Delta t_{warm} + \Delta t_{recon} \leq \Delta t_{mem}. \quad (54)$$

The worst-case warm-up time Δt_{warm} represent the cost of recomputing the new prefix $T_T^\mathcal{E}$ from scratch ($i^* = 1$). From (25):

$$\Delta t_{warm} \propto |T_t^\mathcal{E}|^2 = (\epsilon \cdot |T_t|)^2 \quad (55)$$

where ϵ is the ratio of tokens remaining after eviction $\mathcal{E}(T_t)$.

Reconciliation time Δt_{recon} represents the cost of extending $T_t^\mathcal{E}$ by the tokens accumulated during warm-up:

$$\Delta N = \dot{X} \cdot \Delta t_{warm}, \quad (56)$$

where $\dot{X}$ is the rate new tokens are appended to T_t on $\mathcal{R}_1$. The time to extend T_t^ε is given by (25) with $i^* = |T_t^\varepsilon| + 1$:

$$\begin{aligned} \Delta t_{recon} &\propto \sum_{i=|T_t^\varepsilon|+1}^{|T_t^\varepsilon|+\Delta N} i \\ &= \frac{(|T_t^\varepsilon| + \Delta N - (|T_t^\varepsilon| + 1) + 1)(|T_t^\varepsilon| + \Delta N + (|T_t^\varepsilon| + 1))}{2} \\ &= \frac{\Delta N(2|T_t^\varepsilon| + \Delta N + 1)}{2} \propto \Delta N(2|T_t^\varepsilon| + \Delta N + 1). \end{aligned} \quad (57)$$

The time to out of memory failure Δt_{mem} once eviction is triggered is

$$\Delta t_{mem} = \frac{N_{buffer}}{\dot{X}} = \frac{N_{max} - |T_t|}{\dot{X}} \quad (58)$$

with N_{buffer} representing the number of free space tokens.

Substituting (55), (57), and (58) into (54):

$$(\epsilon \cdot |T_t|)^2 + \Delta N(2|T_t^\varepsilon| + \Delta N + 1) \leq N_{buffer}/\dot{X} \quad (59)$$

Since we can always set the free variables ϵ and N_{buffer} such that the inequality holds, the construction is proved to be a viable and necessary method to satisfy all three theorems. ■

E. Proof for Corollary 1

Proof: Theorem 1 states that the primary objective (19) is optimized when strategy S results in optimal prefix stability (28). The real-time constraint (20) must be satisfied by the optimal prefix stability, or no valid strategy exists (e.g., τ_{max} is set too low for given hardware and sequence

length $|T_n|$). Continuous real-time operation is possible for S implementing asynchronous state reconciliation as defined in Theorem 4.

Theorem 2 states that the information constraint (21) is satisfied for any S that is incrementally extensible (29).

A strategy S providing the eviction (30) in Theorem 3 together with the reconciliation method in Theorem 4 satisfies the finite memory constrain (22).

As (19) is optimized and all constraints (20)-(22) are satisfied, the strategy S must be the optimal strategy S^* . ■

F. Concurrent Request Routing ASR Algorithm

Algorithm 2 provides the complete per-query inference routing procedure as implemented in the real-world experiments, extending Algorithm 1 to the concurrent request setting. While Algorithm 1 describes the background state management logic (buffering, warm-up, catch-up, and the atomic swap), it abstracts over the question of how individual inference queries are served during the reconciliation window. Algorithm 2 addresses this by introducing explicit sequence versioning (e.g. k , k_t) to classify each incoming query into one of three routing cases: (1) normal continuation on the primary resource, (2) reconciliation-phase bridging where the query is served on the primary resource using a stitched context X_{recon} constructed from the current sequence extended with chunks received since eviction, and (3) straggler routing where delayed queries referencing the previous sequence version are served on the secondary resource, which retains its intact KV cache until the next eviction cycle.

Algorithm 2 Asynchronous State Reconciliation with Concurrent Request Routing

Require: Stream of states X_t , reconciliation catch-up threshold $N_{catch-up}$

State Variables:

```

 $j \leftarrow 0$  // Current active sequence static state char length
 $k \leftarrow 0$  // Current active sequence version; current sequence is  $k$ , new evicted sequence is  $k + 1$ 
 $k_{ready} \leftarrow 0$  // Highest sequence version fully precomputed on secondary resource
 $j_\varepsilon \leftarrow \emptyset$  // Evicted sequence static state char length
 $X_{recon} \leftarrow \emptyset$  // Reconciliation state: Extends current sequence with chunks from new evicted sequence
 $\Delta X_\varepsilon \leftarrow \emptyset$  // Evicted state chunk buffer used during catch-up reconciliation phase
 $r_{primary} \leftarrow \mathcal{R}_1$  // Currently active resource with KV cache computed for current sequence
 $r_{secondary} \leftarrow \mathcal{R}_2$  // Resource used to pre-compute the KV cache for the new evicted sequence
is_reconciling  $\leftarrow$  false // Status flag
 $\mathcal{L}_{\Delta X_\varepsilon} \leftarrow \text{Mutex}()$  // Lock for the catch-up buffer

1: Procedure INFERENCE( $X_t$ ):
2:    $j_t, k_t, j_{\varepsilon,t} \leftarrow X_t$  // Unpack state metadata
3:   if  $j_\varepsilon = \emptyset$  then
4:      $j_\varepsilon = j_{\varepsilon,t}$  // State char length of old chunks in evicted states (e.g. excluding new chunks)
5:   end if
6:   // Case 1: Sequence continuation
7:   if  $k_t = k$  then
8:     if  $j_t > j$  then
9:        $X_{recon} \leftarrow X_t[:j_t]$  // For use during reconciliation

```

```

10:    $j \leftarrow j_t$ 
11: end if
12:   LLM( $X_t, r_{\text{primary}}$ )
13: end if
14: // Case 2: Reconciliation
15: if  $k_t > k$  then
16:   if  $k_t > k_{\text{ready}}$  then
17:     //  $r_{\text{secondary}}$  has not finished precomputation  $\rightarrow$  Bridge state and run on  $r_{\text{primary}}$ 
18:      $\Delta X_t \leftarrow X_t[j_\varepsilon : j_t]$  // Extract new chunks from evicted state
19:      $X_{\text{recon}} \leftarrow X_{\text{recon}} \oplus \Delta X_t$  // Append chunks to reconciliation state
20:     Lock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
21:      $\Delta X_\varepsilon \leftarrow \Delta X_\varepsilon \oplus \Delta X_t$  // Append chunks to catch-up buffer
22:     Unlock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
23:      $X_{\text{dyn}} \leftarrow X_t[j_t : ]$  // Extract dynamic suffix (incl. task instruction)
24:      $X'_t \leftarrow X_{\text{recon}} \oplus X_{\text{dyn}}$  // Construct task query based on extended current state sequence
25:      $j_\varepsilon \leftarrow j_t$  // Update eviction cursor
26:     LLM( $X'_t, r_{\text{primary}}$ )
27:   else if  $k_t = k_{\text{ready}}$  then
28:     //  $r_{\text{secondary}}$  has finished precomputation  $\rightarrow$  Resource swap
29:      $r_{\text{primary}}, r_{\text{secondary}} \leftarrow r_{\text{secondary}}, r_{\text{primary}}$  // Swap primary and secondary resource pointers
30:     is_reconciling  $\leftarrow$  false
31:      $X_{\text{recon}} \leftarrow X_t[: j_t]$  // For use during straggler queries
32:      $j, k, j_\varepsilon \leftarrow j_t, k_t, \emptyset$  // Update state cursors and run inference on new sequence
33:     LLM( $X_t, r_{\text{primary}}$ )
34:   end if
35: end if
36: // Case 3: Straggler queries
37: if  $k_t < k$  then
38:   if is_reconciling then
39:     // Protect  $r_{\text{secondary}}$  from significantly delayed requests
40:      $X_{\text{dyn}} \leftarrow X_t[j_t : ]$ 
41:      $X'_t \leftarrow X_{\text{recon}} \oplus X_{\text{dyn}}$ 
42:     LLM( $X'_t, r_{\text{primary}}$ )
43:   else
44:     LLM( $X_t, r_{\text{secondary}}$ ) // Secondary resource holds the intact KV cache for the old sequence
45:   end if
46: end if
47:
48: Procedure RECONCILE( $X_\varepsilon, k_{\text{target}}$ ):
49:   is_reconciling  $\leftarrow$  true // Lock  $r_{\text{secondary}}$  from stragglers
50:   Lock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
51:    $\Delta X_\varepsilon \leftarrow \emptyset$ 
52:   Unlock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
53:   LLM( $X_\varepsilon, r_{\text{secondary}}$ ) // Precompute KV cache on secondary resource with evicted state
54:
55: // Reconcile buffered chunks until sufficiently caught up
56: // NOTE: Reminder is computed at next task inference
57: while  $|\Delta X_\varepsilon| > N_{\text{catch-up}}$  do
58:   Lock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
59:    $\Delta X'_\varepsilon \leftarrow \Delta X_\varepsilon$  // Copy and empty buffer for appending new chunks while precomputing KV cache
60:    $\Delta X_\varepsilon \leftarrow \emptyset$ 
61:   Unlock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
62:    $X_\varepsilon \leftarrow X_\varepsilon \oplus \Delta X'_\varepsilon$ 
63:   LLM( $X_\varepsilon, r_{\text{secondary}}$ )
64: end while
65: Lock( $\mathcal{L}_{\Delta X_\varepsilon}$ )
66:  $\Delta X_\varepsilon \leftarrow \emptyset$ 

```

ACKNOWLEDGMENT

Robin Karlsson was partially supported by grant JPNP23003, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

REFERENCES

- [1] M. Ahn *et al.*, “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” in *CoRL*, 2022.
- [2] W. Huang, *et al.*, “Inner Monologue: Embodied Reasoning through Planning with Language Models,” in *CoRL*, 2022.
- [3] J. Liang, *et al.*, “Code as Policies: Language Model Programs for Embodied Control,” in *ICRA*, 2023.
- [4] D. Driess, *et al.*, “PaLM-E: An Embodied Multimodal Language Model,” in *ICML*, 2023.
- [5] G. Wang, *et al.*, “Voyager: An Open-Ended Embodied Agent with Large Language Models,” *arXiv:2305.16291*, 2023.
- [6] B. Zitkovich *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, 2023.
- [7] C. Packer, *et al.*, “MemGPT: Towards LLMs as Operating Systems,” in *ICML*, 2024.
- [8] G. Xiao, *et al.*, “Efficient Streaming Language Models with Attention Sinks,” in *ICLR*, 2024.
- [9] Y. Liu, *et al.*, “CacheGen: KV Cache Compression and Streaming for Fast Context Switching in LLM Serving,” in *SIGCOMM*, 2024.
- [10] Octo Model Team, *et al.*, “Octo: An Open-Source Generalist Robot Policy,” in *RSS*, 2024.
- [11] M. J. Kim, *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *CoRL*, 2024.
- [12] W. Kwon, *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [13] I. Gim, *et al.*, “Prompt Cache: Modular Attention Reuse for Low-Latency Inference,” in *MLSys*, 2024.
- [14] W. Wang, *et al.*, “Augmenting Language Models with Long-Term Memory,” in *NeurIPS*, 2023.
- [15] Z. Wang, *et al.*, “Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents,” in *NeurIPS*, 2023.
- [16] S. Yao, *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *ICLR*, 2023.
- [17] N. Shinn, *et al.*, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *NeurIPS*, 2023.
- [18] K. Rana, *et al.*, “SayPlan: Grounding Large Language Models using 3D Scene Graphs for Scalable Robot Task Planning,” in *CoRL*, 2023.
- [19] K. Nottingham, *et al.*, “Do Embodied Agents Dream of Pixelated Sheep?: Embodied Decision Making using Language Guided World Modelling,” in *ICML*, 2023.
- [20] B. Liu, *et al.*, “LLM+P: Empowering Large Language Models with Optimal Planning Proficiency,” *arXiv:2304.11477*, 2023.
- [21] A. Zeng, *et al.*, “Socratic Models: Composing Zero-Shot Multimodal Reasoning with Language,” in *ICLR*, 2023.
- [22] Open X-Embodiment Collaboration, *et al.*, “Open X-Embodiment: Robotic Learning Datasets and RT-X Models,” in *ICRA*, 2024.
- [23] D. Zhang, *et al.*, “Large Language Models Are Semi-Parametric Reinforcement Learning Agents,” in *NeurIPS*, 2023.
- [24] Y. J. Ma, *et al.*, “EUREKA: Human-Level Reward Design via Coding Large Language Models,” in *ICLR*, 2024.
- [25] Y. J. Ma, *et al.*, “DrEureka: Language Model Guided Sim-To-Real Transfer,” in *RSS*, 2024.
- [26] I. Singh, *et al.*, “ProgPrompt: Generating Situated Robot Task Plans using Large Language Models,” in *ICRA*, 2023.
- [27] Z. Mandi, S. Jain, and S. Song, “RoCo: Dialectic Multi-Robot Collaboration with Large Language Models,” in *ICRA*, 2024.
- [28] Q. Gu, *et al.*, “ConceptGraphs: Open-Vocabulary 3D Scene Graphs for Perception and Planning,” in *RSS*, 2024.
- [29] D. Shah, *et al.*, “LM-Nav: Robotic Navigation with Large Pre-Trained Models of Language, Vision, and Action,” in *CoRL*, 2022.
- [30] C. Huang, *et al.*, “Visual Language Maps for Robot Navigation,” in *ICRA*, 2023.
- [31] B. Chen, *et al.*, “SpatialVlm: Endowing vision-language models with spatial reasoning capabilities,” in *CVPR*, 2024.
- [32] P. Mariano, N. Di Palo, and E. Johns, “Keypoint action tokens enable in-context imitation learning in robotics,” in *RSS*, 2024.
- [33] W. Huang, *et al.*, “Voxposer: Composable 3d value maps for robotic manipulation with language models,” in *CoRL*, 2023.
- [34] C. Wang *et al.*, “Memento: Fine-tuning LLM agents without fine-tuning LLMs,” in *ICLR*, 2025.
- [35] W. Zhou, *et al.*, “RecurrentGPT: Interactive Generation of (Arbitrarily) Long Text,” *arXiv:2305.13304*, 2023.
- [36] J. S. Park, *et al.*, “Generative Agents: Interactive Simulacra of Human Behavior,” in *UIST*, 2023.
- [37] Y. Zhao, *et al.*, “AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications,” *arXiv:2602.22769*, 2026.
- [38] E. Frantar, *et al.*, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers,” in *ICLR*, 2023.
- [39] J. Lin, *et al.*, “AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration,” in *MLSys*, 2024.
- [40] H. Wang, *et al.*, “BitNet: 1-bit Pre-training for Large Language Models,” *JMLR*, 2025.
- [41] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *ICML*, 2023.
- [42] T. Cai, *et al.*, “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads,” in *ICML*, 2024.
- [43] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *ICLR*, 2024.
- [44] H. Liu and P. Abbeel, “Ring Attention with Blockwise Transformers for Nearly Infinite Context,” in *ICLR*, 2024.
- [45] G.-I. Yu, *et al.*, “Orca: A Distributed Serving System for Low-Latency Large Language Model Inference,” in *OSDI*, 2022.
- [46] L. Zheng, *et al.*, “Efficiently Programming Large Language Models in SGLang,” in *ASPLOS*, 2024.
- [47] T. Pham, *et al.*, “ChunkAttention: Efficient Self-Attention with Arbitrary Granularity,” *arXiv:2402.11131*, 2024.
- [48] NEDO, “Research and development project of the enhanced infrastructures for post-5g information and communication systems,” https://www.nedo.go.jp/english/activities/activities_ZZJP.100172.html, 2024, accessed: 2026-04-07.